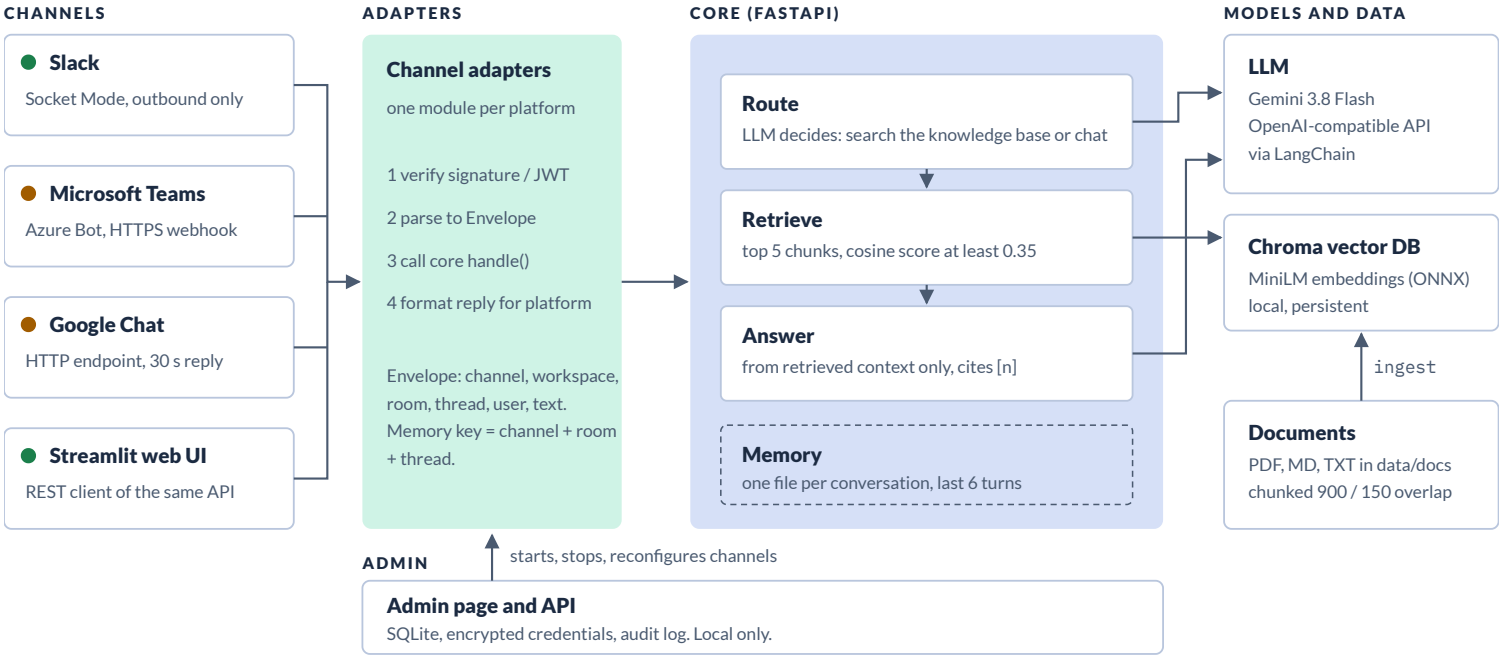
ARCHITECTURE ONE-PAGER

Channel-Agnostic Agent Architecture

One knowledge-base question-answering core. Slack, Microsoft Teams and Google Chat plug into it through thin adapters. Channels and their credentials are added, tested and switched on from an admin page, so changing a channel needs configuration, not changes to the answering code.

System



One question, end to end

1. **Message arrives.**
   The adapter checks Slack's connection, a Microsoft token or a Google token, and turns the event into a common Envelope.
2. **Route.**
   The model decides whether the message needs a lookup. Greetings and small talk skip retrieval; anything unsure searches. Follow-ups are rewritten into standalone queries.
3. **Retrieve.**
   Chroma returns the closest chunks of the indexed documents. Nothing above the score threshold means a fixed "couldn't find this" answer and no model call.
4. **Answer.**
   The model answers only from those chunks and cites them, with a fixed scope and identity prompt that treats document text as untrusted.
5. **Reply.**
   The adapter formats the text for the platform, adds a Sources line (file and page), and sends it back. The turn is saved to memory.

Technology

LAYER	CHOICE
Language	Python 3.11
API	FastAPI, Uvicorn, Pydantic
LLM	Gemini 3.8 Flash through the OpenAI-compatible endpoint; any OpenAI-compatible router works by changing `.env`
Orchestration	LangChain (core, openai, text-splitters)
Vector store	ChromaDB, persistent, cosine similarity
Embeddings	Chroma default, all-MiniLM-L6-v2 (ONNX, runs locally)
Ingestion	pypdf plus recursive text splitter
Slack	slack-bolt, Socket Mode
Teams	Bot Framework SDK (botbuilder-core), Azure Bot
Google Chat	google-auth token verification, HTTP endpoint
Web UI	Streamlit
Admin	FastAPI routes plus a plain HTML/CSS/JS page (no build step); SQLite through SQLAlchemy
Credentials	Fernet encryption (cryptography), master key in `.env`; scrypt password hashes; signed session cookies
Config	Admin database; `channels.yaml` and `.env` import once on first start
Run	`start.ps1` locally; Dockerfile and compose included

Status

- **Slack** — Verified end to end with a real workspace.
- **Teams** — Built and tested offline; needs an Azure Bot and an HTTPS URL.
- **Google Chat** — Built and tested offline; needs Google Workspace and an HTTPS URL.
- **Admin** — Built, 55 tests pass, checked in a browser. First admin account still to be created.
- **Docker** — Not updated for the admin database; not run.

Before production: authentication on the chat API, rate limits, Docker setup for the admin database, a database in place of per-conversation files, pinned dependencies.

Source: github.com/sarthakbobde/channel-agnostic-agent